OpenSWMM Uncertainty Sidecar — User Guide

OpenSWMM Uncertainty Sidecar — User Guide

> **Status (2026-05-25):** Phases 0–9 complete.
> 45 unit tests + 6 regression tests + 2 engine integration tests pass.
> The 2D spectral ROM (scalar + spatial), coupling uncertainty, runoff ensemble, WQ bounds, Fiedler diagnostics, the 1D spectral ROM struct, and the 1D ROM engine lifecycle are all implemented and tested. HDF5 quantile output is the one remaining gap.

Contents

---

1. What this is

The uncertainty sidecar runs an M-member parameter ensemble **alongside** the deterministic CVODE 2D solver — not instead of it — at roughly 1000× lower cost than full Monte Carlo. After each routing step you get three additional per-cell depth fields:

Field	Meaning
`q05`	5th-percentile depth across the ensemble
`q50`	Median depth (best single estimate)

Field	Meaning
`q95`	95th-percentile depth

Together `[q05, q95]` is a **90th-percentile prediction interval**: if the true Manning's n and rainfall intensity lie within the perturbation ranges you specify, there is roughly a 90 % probability that the true depth at each cell lies inside the band.

What parameters can be uncertain:

Layer	Parameter	Section
2D surface	Manning's n (scalar or spatially-correlated)	`[2D_ROM]` + `[UNCERTAINTY]`
2D surface	Rainfall intensity (scalar or spatially-correlated)	`[2D_ROM]` + `[UNCERTAINTY]`
2D coupling	Discharge coefficient Cd (inlet grates)	`[UNCERTAINTY]`
1D sewer	Manning's n (pipe roughness)	`[UNCERTAINTY]`
1D sewer	Lateral inflow / DWF rate per node	`[UNCERTAINTY]`
Runoff	Soil hydraulic conductivity (Green-Ampt Ks, Horton f0/fmin, CN S)	`[UNCERTAINTY]`

2. Quick start

Add `[2D_ROM]` and `[UNCERTAINTY]` to your `.inp` file. The `[UNCERTAINTY]` section overrides the perturbation values in `[2D_ROM]`, so you only need one of the two for a basic run.

```
[2D_ROM]
ENABLE        YES
MEMBERS       50
MODES         10

[UNCERTAINTY]
;;Layer  Parameter    Perturbation
2D       MANNINGS_N   0.20
2D       RAINFALL     0.10
```

That's it. After the simulation, read quantiles from the engine (see §2.1) or from the future HDF5 output file (§7.1).

2.1 Reading quantiles (current API)

Quantiles are accessed via the C++ API after `swmm_engine_step`:

```c
#include "openswmm/engine/openswmm_engine.h"
#include "core/SWMMEngine.hpp"
#include "2d/uncertainty/SpectralROM.hpp"

SWMM_Engine handle = swmm_engine_create();
swmm_engine_open(handle, "model.inp", "model.rpt", nullptr, nullptr);
swmm_engine_initialize(handle);
swmm_engine_start(handle, 0);

double elapsed = 1.0;
while (elapsed > 0.0)
    swmm_engine_step(handle, &elapsed);

auto* eng = static_cast<openswmm::SWMMEngine*>(handle);
const openswmm::twoD::SpectralROM* rom =
    eng->surfaceRouter2D().rom();   // null if ROM inactive or not yet
        seeded

if (rom && rom->is_ready()) {
    // rom->q05, rom->q50, rom->q95 — length n_tri each
    for (int i = 0; i < n_tri; ++i)
        printf("cell %d: q50 = %.4f m  [%.4f, %.4f]\n",
               i, rom->q50[i], rom->q05[i], rom->q95[i]);
}
```

The cast is safe: `SWMM_Engine` is `typedef void*` and is always `new SWMMEngine()` stored as a pointer. All C++ symbols are exported (`CXX_VISIBILITY_PRESET default`).

---

3. Input file reference

3.0 `[2D_ROM]` vs `[UNCERTAINTY]` — what each section controls

These two sections serve different purposes and can be used together or independently.

Section	Controls	Does NOT control
`[2D_ROM]`	*How* the ROM runs: ensemble size M, modes k, K_eff, spatial correlation lengths, reseed thresholds, parametric tails	Perturbation levels; which parameters are uncertain
`[UNCERTAINTY]`	*What* is uncertain and by how much: which physical parameters carry a prior, the perturbation half-range, and which model layer (1D/2D)	ROM mechanics

Think of `[2D_ROM]` as the **solver configuration** (accuracy, cost, spatial resolution) and `[UNCERTAINTY]` as the **experiment configuration** (what you don't know and by how much).

How they interact: - A `2D MANNINGS_N 0.20` entry in `[UNCERTAINTY]` overrides `MANNINGS_PERT` in `[2D_ROM]` and also implicitly enables the ROM (`ENABLE YES`). - If both sections are present, `[2D_ROM]` sets the ROM mechanics; `[UNCERTAINTY]` overrides only the perturbation levels. - You do not need both. For a minimal run: `[UNCERTAINTY]` `2D MANNINGS_N 0.20` is sufficient.

1D sewer ROM: there is no `[1D_ROM]` section. The 1D sewer ROM is activated exclusively by a `1D` layer entry in `[UNCERTAINTY]`. A pure 1D model (no 2D mesh) only needs `[UNCERTAINTY]`.

3.1 `[2D_ROM]`

Controls the ROM solver. All keywords are optional; defaults shown.

Keyword	Default	Range	Description
`ENABLE`	`NO`	YES/NO	Activate the ROM sidecar.
`MEMBERS`	`50`	≥ 2	Ensemble size M. More members → smoother quantiles, O(M) cost.
`MODES`	`10`	≥ 1	Laplacian eigenmodes retained for the ROM basis. More modes → finer spatial resolution of spread. Capped at min(MEMBERS, n_tri−1).
`MANNINGS_PERT`	`0.20`	≥ 0	Half-range for Manning's n: each member's n ∈ [1−p, 1+p] × n_base. Overridden by `[UNCERTAINTY]` `2D MANNINGS_N`.
`RAINFALL_PERT`	`0.20`	≥ 0	Half-range for rainfall intensity. Overridden by `[UNCERTAINTY]` `2D RAINFALL`.
`K_EFF`	`≤ 0 → AUTO`	any	Effective diffusive conductance

Keyword	Default	Range	Description
			(m^(4/3)/s). Values ≤ 0 activate AUTO mode (see §4.8).
MANNINGS_CORR_LEN	0.0	≥ 0	Spatial correlation length (m) for Manning's n field. 0 = uniform scalar per member (fast).
RAINFALL_CORR_LEN	0.0	≥ 0	Spatial correlation length (m) for rainfall field. 0 = uniform scalar per member.
WET_RESEED_FRACTION	0.05	[0, 1]	Trigger a ROM reseed when the wet-cell count changes by more than this fraction of n_tri. Prevents stale seeds on rapidly advancing wetting fronts.
WET_RESEED_MIN_INTERVAL	60.0	≥ 0	Minimum simulation time (s) between consecutive reseeds. Prevents excessive reseeding on oscillating fronts.
PARAMETRIC_TAILS	NO	YES/NO	Fit a log-normal to the wet-member sub-population and use the analytic 95th percentile as q95 (reduces noise from top 1–2 samples when M is small). q05/q50 remain sort-based.

Keyword	Default	Range	Description
MODE_DROP_THRESHOLD	1e-10	≥ 0	Drop mode j from advance() when its energy E_j and rainfall forcing are both below this threshold. Automatically reactivated by rainfall.

3.2 [UNCERTAINTY]

Format: LAYER PARAMETER [DISTRIBUTION] PERTURBATION

- DISTRIBUTION is optional; defaults to UNIFORM.
- Supported distributions: UNIFORM, NORMAL, LOGNORMAL.
- [UNCERTAINTY] entries **override** the corresponding scalar fields in [2D_ROM].

Currently parsed parameters:

```
[UNCERTAINTY]
;;Layer  Parameter    [Distribution]  Perturbation
2D       MANNINGS_N                   0.20    ;; overrides [2D_ROM]
         MANNINGS_PERT
2D       RAINFALL     UNIFORM         0.15    ;; same effect as
         RAINFALL_PERT
1D       MANNINGS_N                   0.20    ;; enables 1D sewer ROM
         (see §7.7)
1D       RAINFALL                     0.20    ;; lateral-inflow
         uncertainty for 1D
```

LAYER is either 2D (surface routing) or 1D (sewer network routing). Both layers accept MANNINGS_N and RAINFALL with any supported distribution.

A 2D entry also enables and configures the 2D ROM (sets enable_rom = true in [2D_ROM]). A 1D entry only populates the uncertainty config — the 1D ROM is built automatically in initialize() whenever any active source is present, or whenever the 2D ROM is enabled (so 2D→1D coupling always sees per-member 1D heads).

> **Tip — pure 1D (no 2D mesh):** adding 1D MANNINGS_N 0.20 to a plain 1D .inp is sufficient; no [2D_ROM] or [2D_MESH] sections are needed.

Support for SOIL (runoff ensemble) and CD (coupling-flux uncertainty) are set via C++ API directly (parsed by UncertaintyEnsemble, not by the .inp handler).

3.3 [2D_OPTIONS] — solver settings

These govern CVODE accuracy, not the ROM. Set tighter tolerances for small or stiff meshes.

Keyword	Default	Description
MAX_TIMESTEP	10.0	Max CVODE internal step (s)
MIN_TIMESTEP	0.001	Min CVODE internal step (s)
REL_TOLERANCE	1e-4	Relative error tolerance
ABS_TOLERANCE	1e-6	Absolute error tolerance (m)
DRY_DEPTH	0.001	Cells below this depth (m) are treated as dry
MAX_CVODE_STEPS	500	Max CVODE steps per SWMM routing interval
COUPLING_CD	0.65	Default discharge coefficient for 2D↔1D inlets

4. How it works

4.1 The operator's problem

An engineer has just finished calibrating a SWMM model for a city drainage review. The deterministic run takes 4 minutes. The report is due tomorrow. Everything seems set — until the question arises: *what is Manning's n for that mixed concrete-and-gravel surface in the north-east catchment?*

The calibration data from 2019 suggest 0.015. The textbook range is 0.012–0.018. Field inspection says "somewhere in there." If the true value is 20% higher than assumed, how much deeper does flooding get at the critical junction? How confident should the engineer be in the 100-year return level?

The traditional answer: run 50 simulations with n perturbed across the range. Cost: 200 minutes. The engineer does not have 200 minutes.

What most engineers do instead: run one simulation and add a caveat — "Manning's n is uncertain, results may vary." This is honest but quantifies nothing.

What the ROM sidecar does: run 50 virtual members as a lightweight parallel process alongside the single deterministic solver. Extra cost: about 2 seconds on a 4-minute model. At every node or cell, at every timestep, you get three additional fields: `q05` (optimistic scenario), `q50` (best estimate), `q95` (pessimistic scenario). Together they form a **90th-percentile prediction interval** — if the true Manning's n is anywhere within ±20% of the calibrated value, the depth lies inside the band for roughly 90% of timesteps.

4.2 Why eigenmodes — the guitar string analogy

A guitar string vibrates in discrete modes: the fundamental, the octave, the fifth above that. Each mode has a characteristic frequency and a characteristic spatial shape. To describe how the string responds to any perturbation you do not need to track every atom

— you only need the amplitudes of the first few modes. Higher modes decay almost instantly; only the low-frequency modes persist over timescales of interest.

Water surfaces in drainage networks behave the same way. The "shape modes" of a 2D mesh — or a 1D pipe-network graph — are the eigenvectors of the **graph Laplacian** L. The Laplacian encodes which cells or nodes are connected and how strongly. Its eigenvectors are the natural decay modes of any diffusing quantity on that network:

- **Mode 0 (null mode)**: a uniform depth shift — it does not spread and is excluded.
- **Mode 1 (Fiedler mode)**: the smoothest non-trivial mode; it identifies the network's tightest hydraulic bottleneck (see §7.4 for the diagnostic).
- **Higher modes**: increasingly fine-grained spatial patterns that decay faster and faster.

When Manning's n is uncertain, the **uncertainty in the water surface** is well-captured by the first k eigenmodes. A member with high Manning's n evolves differently from a member with low n — but both evolve in the same low-dimensional subspace spanned by those k modes. Retaining k=10 modes out of n=10,000 cells reduces the per-member state from 10,000 numbers to 10 — a 1000× compression of state space.

4.3 The key elegance — an exact solve, no timestep limit

After projecting onto the eigenbasis, the linearised diffusion-wave PDE becomes k **independent scalar ODEs**, one per retained mode per ensemble member:

```
da[i,j]/dt = −rate[i,j] · a[i,j]  +  f[i,j]

rate[i,j] = λ_j · K_eff / mannings_mult[i]   (mode j decays faster for
smoother channels)
f[i,j]    = r_coarse[j] · rainfall_mult[i]   (rainfall projected onto
mode j)
```

Each ODE has an **exact analytical solution** — the exponential integrator:

```
a[i,j](t+dt) = (a[i,j](t) − f[i,j]/rate[i,j]) · exp(−rate[i,j] · dt)
+  f[i,j]/rate[i,j]
```

This is not an approximation. It is the exact solution for the linearised system at any `dt`. There are no substep limits, no Newton iterations, no Krylov solves. Advancing the entire M-member ensemble requires M×k multiplications and exponentials — roughly 500 floating-point operations for M=50, k=10. A modern CPU executes this in under 1 microsecond.

Compare with the deterministic CVODE solver, which performs many nonlinear RHS evaluations on all n cells per routing step. The ROM sidecar costs less than 0.1% of total simulation time.

4.4 Why this is new

SWMM is 50 years old. Uncertainty quantification in stormwater modelling has been studied for decades. So why does this feel new?

Previous methods run outside the solver. Monte Carlo, FOSM (first-order second-moment), and Morris sensitivity screening all require running the full solver N times and comparing outputs. They are expensive, disconnected from the solver's internal state, and produce no per-timestep uncertainty bands without storing complete simulation ensembles. The ROM sidecar runs *inside* the solver, advancing its ensemble in lock-step with the deterministic solution, with access to the current depth field at every routing step.

ROM ideas existed but not for this problem. Galerkin and POD-based ROMs have been applied to river and coastal shallow-water equations since the early 2000s — but almost exclusively as offline surrogates: build a surrogate from a library of full-model snapshots, then query it instead of the full model for new inputs. What is different here is the specific combination: (1) a *graph-Laplacian eigenbasis* computed once from mesh topology (no snapshot library needed), (2) running as a *live sidecar* inside the solver rather than replacing it, and (3) handling irregular urban networks with wet/dry transitions and coupled 1D/2D domains. That combination has not been applied to SWMM-class urban drainage before.

The engine architecture had to catch up first. The ROM's inner loop iterates over all N node depths for every ensemble member on every routing step — a tight numerical loop that is sensitive to memory layout. SWMM5's object-oriented design stored node data as arrays of structs (each node object holding all its fields). Iterating over depths meant pointer-chasing through non-contiguous memory: cache-unfriendly at scale. SWMM6's data-oriented restructuring (struct-of-arrays: `nodes.head[]`, `nodes.depth[]`, `links.roughness[]` as flat contiguous arrays) was a prerequisite for the ROM sidecar to be fast enough to run at every timestep on large networks. The data layout change that looks like an internal refactor is also what makes the 1000× speedup practically achievable.

The 1000× speedup crosses a practical threshold. At 10× speedup, uncertainty quantification is marginally better than Monte Carlo. At 100×, it is practical for research. At 1000×, it is operationally free: a 4-minute simulation gains a 90th-percentile prediction band for under 0.25 seconds of additional compute. That threshold is what makes this useful in everyday engineering practice, not just in papers.

The remaining subsections give the precise mathematical definitions for implementers.

4.5 ROM setup and lifecycle

The 2D and 1D ROMs are built and seeded independently at `initialize()`, then advance alongside their respective solvers at every routing step.

2D ROM — what runs when

At `initialize()` — mesh Laplacian built, eigenbasis computed, ROM allocated:

The 2D mesh Laplacian is assembled from triangle connectivity (shared-face lengths, centroid distances, cell areas). The Lanczos+QL eigensolver extracts the k lowest non-trivial eigenvectors P[:,0..k−1] and eigenvalues λ_0..λ_{k-1} from this Laplacian. If `SpectralPrecond2D` is also active, the ROM reuses its already-computed eigenbasis rather than rerunning the solver. A `SpectralROM` struct is allocated with an M×k coefficient matrix (all zeros) and an LHS parameter design for M members.

At the first `CvodeSurfaceSolver::advance()` call — seeding:

`seedROM()` projects the current CVODE depth array h onto the eigenbasis:

```
a[i,j] = P[:,j]ᵀ · h    for all members i = 0..M−1
```

Every member starts from the same deterministic IC, so spread = 0 at seed time. The different LHS multipliers (`mannings_mult[i]`, `rainfall_mult[i]`) give each member a different decay rate and forcing on the very next `advance()` call, so spread grows immediately.

At each subsequent CVODE step — advance and quantiles:

1. `applyCouplingFluxToROM()` injects per-member drainage fluxes at coupling points (if any; see §4.11).
2. `SpectralROM::advance(dt, K_eff, rainfall)` updates all M×k coefficients via the exponential integrator.
3. `computeQuantiles()` reconstructs per-cell depths for all M members and sorts to get q05/q50/q95.

Reseeding — keeping the ROM aligned with a moving wetting front:

If the wet-cell count changes by more than `WET_RESEED_FRACTION × n_tri` since the last seed, and at least `WET_RESEED_MIN_INTERVAL` seconds have elapsed, the ROM reseeds from the current CVODE depth. Spread briefly collapses to zero and re-grows as members diverge under their different multipliers.

1D ROM — what runs when

At `SWMMEngine::initialize()` — network Laplacian built, eigenbasis computed, ROM seeded:

`buildROM1D()` collects all conduit node pairs from `ctx_.links`. Outfall nodes are identified from `ctx_.nodes.type` and excluded from the active set — they are fixed-head boundaries and cannot carry spreading uncertainty. `NetworkLaplacian1D::buildUniform()` assembles the graph Laplacian in CSR format over the remaining active nodes. `GraphEigenBasis::build()` runs Lanczos+QL to extract k eigenvectors and eigenvalues. The ROM is then seeded immediately from the current hydraulic state:

```
a[i,j] = P[:,j]ᵀ · h_active    for all members i = 0..M−1
```

where `h_active` contains only the heads of active (non-outfall) nodes in the ROM's index space. The `full_to_active[]` map (indexed by SWMM node index) records which active ROM index corresponds to each node, and returns −1 for outfalls.

At each `stepRouting()` call — advance and quantiles:

1. `computeK1d()` estimates mean diffusion-wave diffusivity from current conduit depths, slopes, and roughnesses, then normalises by conduit length² to yield K1d [1/s].
2. `SpectralROM1D::advance(dt, K1d, nullptr)` updates all M×k coefficients.
3. `computeQuantiles()` reconstructs per-active-node head quantiles.

The 1D ROM is **not automatically reseeded** during a simulation run. It seeds once at `initialize()` from the hydraulic initial condition and evolves through the event from there.

How the sidecar attaches — five hook points

The ROM is a **read-only observer** of the deterministic solver. It never writes to CVODE state or the DYNWAVE solution; the solver drives, the sidecar observes and reports. But it does not attach loosely — it reads from five specific hook points, each of which extracts a quantity the deterministic solver computes anyway:

Hook	What the sidecar reads	How it is used
`initialize()`	Mesh/network topology — triangle connectivity, conduit node pairs, cell areas, conduit lengths	Assembles the Laplacian eigenbasis: the same operator CVODE will invert at every Newton step
First `CvodeSurfaceSolver::advance()`	CVODE depth array `h[]` (current state after solver warmup)	Seeds all M members: `a[i,j] = P[:,j]ᵀ · h`
Each `advance()`	Current wet-cell depths, Manning's n values, bed slopes	Computes K_eff (2D) or K1d (1D) — the same linearisation coefficient the solver uses for its Jacobian
Coupling exchange	Deterministic DYNWAVE node heads from `computeCouplingExchange()`	Per-member orifice equation — M realisations of what the inlet head difference is
Reseed check	CVODE's current wet-cell count	Decides when the seed state has drifted too far from the deterministic

Hook	What the sidecar reads	How it is used
		solution to trust

This is the barnacle design: the ROM attaches to the hull at initialization, moves with the solver through every timestep, draws parameters from quantities the solver already computes, and emits three extra depth fields per cell without altering the solver's own trajectory.

How the two ROMs differ in basis construction

	2D ROM (`SpectralROM`)	1D ROM (`SpectralROM1D`)
Laplacian type	Geometric mesh Laplacian (area/distance-weighted)	Graph (topological) Laplacian (connectivity only, uniform weights)
Eigenvalue character	Carries spatial scale implicitly via mesh geometry	Dimensionless — counts graph connectivity, not metres
Diffusivity parameter	K_eff [m^(4/3)/s] — spatial scale baked into mesh Laplacian	K1d = D/L² [1/s] — must normalise by L² to give 1/s
When seeded	Deferred to first CVODE advance (solver warms up first)	Immediately at `initialize()` (from initial steady-state heads)
Automatic reseeding	Yes — triggered by wetting-front advance	No
Basis shared with preconditioner?	Yes, if `SpectralPrecond2D` active	No — always a standalone `GraphEigenBasis`
Output quantiles sized	n_tri (triangles)	n_active_nodes (excluding outfalls)

4.6 The ROM ODE (full definition)

The 2D diffusion-wave equation (linearised about the current deterministic state) is:

```
∂h/∂t = K_eff · L · h + r(x)
```

where `L` is the graph Laplacian of the triangulation and `r(x)` is the per-cell rainfall rate. Projecting onto the Laplacian eigenbasis `P` (columns = eigenvectors, eigenvalues `λ_j`):

```
da_j/dt = −rate_j(θ) · a_j  +  f_j(θ)

rate_j(θ) = λ_j · K_eff / mannings_mult[i]     (mode-j decay rate for
member i)
f_j(θ)    = r_coarse[j] · rainfall_mult[i]     (mode-j forcing for
member i)
```

```
r_coarse[j] = P[:,j]ᵀ · rainfall_field
```

Each ensemble member carries its own `(mannings_mult, rainfall_mult)` drawn from the LHS design. The ODE is **solved exactly** via the exponential integrator — no substep limit, no Krylov solves:

```
a_j(t+dt) = (a_j(t) − f_j/rate_j) · exp(−rate_j · dt)  +  f_j/rate_j
```

When `rate_j` is near zero (near-null mode or K_eff ≈ 0), the solver falls back to Euler. Reconstructed cell depths are clamped to ≥ 0.

Connection to the CVODE Jacobian:
For the linearised system `∂h/∂t = K_eff · L · h`, the Jacobian is `J = K_eff · L`. Its eigenvalues are exactly `{λ_j · K_eff}` — the same values that appear as the ROM's per-mode decay rates. The ROM basis P is not arbitrary: it diagonalises the operator that CVODE's Newton–Krylov solver is implicitly inverting at each timestep. The k retained modes are the k slowest-decaying directions of the linearised dynamics — the most persistent patterns in the solution space and therefore the ones that carry the most uncertainty at the timescales of interest. Modes with high `λ_j` decay so fast (sub-second) that they contribute negligible uncertainty by the time the CVODE step completes; they are safely discarded.

The same formulation applies to the 1D sewer ROM (`SpectralROM1D`), substituting the network graph Laplacian for the 2D mesh Laplacian. The 1D K_eff is computed as the diffusion-wave diffusivity `D = h^(5/3) / (2n√S)` [m²/s] averaged over active conduits, then normalised by conduit length squared `L²` to yield units of 1/s. This normalisation is essential: graph Laplacian eigenvalues are dimensionless (they count topological connectivity, not spatial scale), so `λ_j × K1d` must have units of 1/s.

Does this work with SWMM's dynamic wave routing?

2D surface domain: The CVODE solver solves the diffusion-wave equation by design — the ROM's basis is derived from the exact same equation, so the Jacobian connection above holds exactly. "Dynamic wave" in the 2D context does not apply.

1D sewer network: SWMM's `DYNWAVE` solver integrates the full **Saint-Venant equations** (continuity + momentum). The 1D ROM propagates uncertainty using a **diffusion-wave approximation** — the momentum equation is dropped. This approximation is valid when inertial terms are small compared to pressure-gradient and friction forces, which is true for: - Subcritical flow in partially-full pipes (Fr ≪ 1) - Gradually-varying, pressure-gradient-dominated events - Most standard urban drainage design storms

It breaks down — and the ROM spread should be treated as a qualitative indicator only — for: - Surcharging or fully-pressurized mains (the momentum balance changes fundamentally) - Rapidly rising hydrographs where `∂Q/∂t` is large - Tidal or strongly backwater-controlled reaches - Pump stations, inline storage, or sluice gates (discontinuous momentum sources)

In these regimes the ROM correctly identifies *which nodes* carry the most Manning's n sensitivity (the spread pattern is qualitatively right) but may mis-estimate the band width by 30–50%. See also §8, limitation 2.

Roadmap: Jacobian-informed 1D ROM basis

SWMM's DYNWAVE Picard iteration already assembles a linearized operator H at each Newton step. The existing `SpectralCoarse` infrastructure extracts it:

```
active_diag[ai] = max(surf_area - 0.5*dt*sumdqdh, min_surf_area)
conduit_off[ci]  = 0.5*dt * dqdh_[tile_uj_[ci]]   // H =
        surfaceArea/dt - dQ/dh
```

H includes inertial terms, actual pipe depths, backwater, and surcharge effects through the live `dqdh` values — it is the true dynamic-wave Jacobian, not a diffusion-wave approximation. Using H's eigenvectors as the ROM basis would make the 1D ROM accurate for the full Saint-Venant regime.

`SpectralCoarse` was closed as unsafe because it *wrote corrections back* to the DYNWAVE state, contaminating `links.flow` across timesteps. A read-only ROM sidecar would avoid this entirely — it reads H's eigenvectors to update its own coefficients and never touches the solver state.

Three open questions before this can be implemented:

1. **Re-eigensolving cost**: H changes as pipe depths change. Recomputing k eigenvectors per step (or every N steps) costs O(N·k) extra. Affordable for small networks; potentially significant at 10k+ nodes. Amortised re-solving (e.g., every 10 routing steps) is a practical middle ground.
2. **Time-varying basis**: when P changes between timesteps, the modal coordinates `a[i,j]` are defined on a new basis. The sidecar must re-project `a_old` onto `P_new` before advancing: `a_new[i,j] = Σ_k (P_new[:,j]ᵀ · P_old[:,k]) · a_old[i,k]`.
3. **Non-symmetry of H**: H is not symmetric due to the convective term `∂(Q²/A)/∂x`. Standard Lanczos (used for the graph Laplacian) requires symmetric input; a one-sided or generalised eigensolver would be needed.

This enhancement would remove limitation 2 from §8 entirely for the 1D domain.

4.7 Latin-hypercube design

`UncertaintyEnsemble` generates four decorrelated LHS columns, each stratifying its parameter uniformly across M strata (midpoint rule):

Column	Ordering	Range	Correlation with Manning
`mannings_mult`	ascending	[1−p_n, 1+p_n]	—
`rainfall_mult`	descending	[1−p_r, 1+p_r]	near-zero (reversed order)
`soil_mult`	Fisher-Yates shuffle of strata	[1−p_s, 1+p_s]	< 0.05 (seed+2)
`cd_mult`	Fisher-Yates shuffle of strata	[1−p_cd, 1+p_cd]	< 0.05 (seed+3)

The reversed-order trick for rainfall gives perfect anti-correlation between Manning and rainfall columns within each realisation — members with high Manning's n (slow conveyance) get low rainfall multipliers, which is physically conservative and prevents artificial spread inflation.

4.8 AUTO K_eff

At the first CVODE advance (and at each reseed), K_eff is estimated from the current wet state:

```
K_eff = h_mean^(5/3) / (2 · n_mean · √S_mean)
```

- `h_mean` — mean depth over cells with depth > 1e-6 m
- `n_mean` — area-weighted mean Manning's n over wet cells
- `S_mean` — mean bed slope magnitude (floor: 1e-6, prevents divide-by-zero on flat domains)

Option A (per-mode Rayleigh quotient): when the advance call receives the current depth array, each mode j gets its own K_eff weighted by the depth distribution:

```
K_eff_j = K_eff · Σ_t φ_j[t]² · (h[t]/h̄)^(5/3)
```

Modes concentrated in deep cells decay faster. This is enabled automatically when h_cell is passed to `advance()` (which `CvodeSurfaceSolver` does by default).

4.9 Quantile computation

After each `advance()` call, `computeQuantiles()` reconstructs per-cell depths for all M members and computes the three percentiles by sorting:

```
h_i[t] = max(0,  Σ_j P[t,j] · a_{i,j})      for member i, cell t
```

The three output fields are the sorted 5th, 50th, and 95th percentile of `{h_i[t] : i=0..M−1}` at each cell `t` independently.

With `PARAMETRIC_TAILS YES`: for cells where ≥ 4 members are wet, fits a log-normal to the wet sub-population and replaces the sort-based q95 with the analytic 95th percentile of that fit. This suppresses noise from the top 1–2 samples when M is small (< 30).

4.10 Spatial uncertainty fields

When `MANNINGS_CORR_LEN > 0` or `RAINFALL_CORR_LEN > 0`, each member gets a spatially-varying multiplier field generated by `CorrelatedFieldGenerator`:

1. Draw i.i.d. N(0,1) samples at each triangle centroid.
2. Smooth with an exponential kernel: `Z_smooth[t] = Σ_{t'} exp(−d(t,t')/L) · Z[t']` (summed over neighbors within `3·L` metres of centroid `t`).
3. Centre the field: `Z_centred[t] = Z_smooth[t] − mean(Z_smooth)`.
4. Combine with the member's global LHS level: `W[i][t] = global_level[i] + pert · Z_centred[i][t]`

As `corr_len → ∞`, spatial variation collapses toward a uniform field (scalar limit). When `corr_len = 0`, the spatial generator is bypassed for speed.

In `advance()`: `f_j^i = P[:,j]ᵀ · (rainfall ⊙ W_rain[i])` replaces the scalar `r_coarse[j] · rainfall_mult[i]` when spatial rainfall is active.

4.11 Coupling uncertainty

When both a 2D ROM and a 1D ROM are active, the coupling exchange operates on the full ensemble rather than on a single deterministic pair of heads. The sequence at each SWMM routing step is:

Step 1 — deterministic exchange (shared heads)

`computeCouplingExchange()` computes a single inflow/outflow at each coupling point using the current CVODE surface depth and the DYNWAVE node head. This updates `ctx_.nodes.lat_flow[]` and drives the deterministic 1D DYNWAVE solve — exactly as in a run with no ROM.

Step 2 — per-member ROM exchange

`applyCouplingFluxToROM()` re-runs the orifice equation independently for all M ensemble members at each non-outfall coupling point:

- **2D head per member**: reconstructed from 2D ROM coefficients: `h_2d[i][ci] = max(0, Σ_j P[ci,j] · a[i,j])`
- **1D head per member**: if the 1D ROM is registered, `h_1d[i] = rom1d->reconstructHead(i, full_to_active[cp.node_idx])`; for outfall nodes (`full_to_active == −1`) the shared deterministic head is used as a fallback.
- **Orifice flow per member**: `Q_i = Cd · A · sign(h_2d[i] − h_1d[i]) · √(2g|h_2d[i] − h_1d[i]|)`, capped by available depth in the 2D cell.
- **2D ROM coefficient update** (for each retained mode j): `a[i,j] += P[j,ci] · (−Q_i · dt / tri_area)`

Step 3 — independent ROM advances

After coupling, `SpectralROM::advance()` and `SpectralROM1D::advance()` run independently for the remainder of the routing step using their respective K_eff and K1d. The coupling flux injected in Step 2 is a one-shot impulse on the 2D ROM coefficients; the 2D ROM then evolves freely until the next routing step.

What the 1D ROM is not updated by:

The 1D ROM coefficients are not modified by `applyCouplingFluxToROM()`. The 1D ROM sees the coupling only indirectly — through the lateral flow term in DYNWAVE, which updates `ctx_.nodes.head[]`, which in turn affects K1d and the initial seed state. This is an operator-splitting approximation: the 2D ensemble absorbs per-member coupling uncertainty; the 1D ensemble evolves under its own Manning uncertainty and contributes per-member 1D heads to the coupling exchange in Step 2.

The per-coupling-point bounds `[q_min, q_max]` across all M members are accumulated in `CouplingUncertaintyOutput` and available via `SurfaceRouter2D::couplingOutput()`.

---

5. How to interpret outputs

5.1 What q50, q05, q95 mean

- **q50** is the median of the ensemble — the best single-number estimate when Manning's n is uncertain. For small perturbations (≤ 20%) it is close to, but not identical to, the deterministic CVODE result (small systematic bias from the linearisation).
- **[q05, q95]** is a 90-percentile prediction interval. It answers: "if the true Manning's n and rainfall are anywhere within the specified ±p% range, what is the range of outcomes?"
- **q95 − q05** (the spread) quantifies where uncertainty matters most. Large spread = high sensitivity to the uncertain parameters at that cell and time.

5.2 Typical spatial patterns

Observation	Likely cause
Spread peaks at inlet/outlet junctions	These are hydraulic bottlenecks where Manning's n most strongly controls conveyance. The Fiedler gradient (§7.4) confirms this.
Spread grows downstream along a pipe	Manning's n uncertainty compounds over flow path length — each cell passes its uncertainty to the next.
Spread is near zero early in the event, grows later	The ROM seeds from a uniform depth field (zero spread). Non-uniform patterns develop as the solver advances, giving the ensemble room to diverge.
Spread is near zero everywhere despite non-zero perturbation	Initial depth is spatially uniform (projects to null Laplacian eigenvector). Use a non-uniform IC or wait for rainfall to build non-uniform depth gradients.
High relative spread in dry cells	Relative spread = (q95−q05)/q50 blows up as q50 → 0. Use **absolute** spread (q95−q05 in metres) for dry-cell analysis.

5.3 Choosing perturbation levels

The perturbation `p` represents the half-range of the assumed uniform prior for the parameter multiplier. Physically:

- **Manning's n ± 20%**: matches typical calibration uncertainty in urban drainage (USEPA 1992 SWMM manual reports ±15–25% for concrete and grass-lined channels). Use 0.15 for well-calibrated systems, 0.25–0.30 for uncalibrated design-phase models.
- **Rainfall ± 10%**: captures radar QPE error at typical urban scales. Use 0.15–0.20 for long-duration events where spatial variability is significant.
- **Soil Ks ± 25%**: typical coefficient of variation for saturated conductivity from point measurements (Rawls & Brakensiek 1989).

The output band width is approximately linear in `p` for small perturbations:

```
q95 − q05 ≈ 2 · p · |∂(depth)/∂(Manning's n)| · n_base
```

5.4 Ensemble size guidance

M	Use case
10–20	Rapid screening runs, qualitative band shape only. q05/q95 noisy.
50	Default. Acceptable quantile accuracy (± ~5% of spread).
100	Required if using `PARAMETRIC_TAILS NO` and need reliable q95 tail.
200+	Diminishing returns; use if publishing or for regulatory submissions.

Cost scales exactly as O(M). Doubling M doubles ROM overhead (which is already ~1% of total simulation time), so M=100 adds roughly 2% overhead relative to M=0.

5.5 Modes guidance

k	Effect
5	Only the coarsest spatial uncertainty patterns captured. Fast, good for large meshes.
10	Default. Captures spatial features down to ~3× the mean triangle edge length.
20–30	Fine-grained spatial uncertainty; diminishing returns for scalar Manning perturbation. More useful when `MANNINGS_CORR_LEN` is small (spatially heterogeneous uncertainty).

The ROM cannot represent spread at spatial scales finer than the kth eigenmode. If the mesh has 10,000 triangles and k=10, the ROM can resolve spread patterns at scales of roughly `domain_size / sqrt(k)` — about 30% of the domain width for a square domain.

5.6 Coupling uncertainty interpretation

`SurfaceRouter2D::couplingOutput()` returns per-coupling-point bounds `{q_min, q_max}` (m^3/s). These represent the range of inlet drainage flows across the ensemble:

- `q_max − q_min` large → the inlet flow is sensitive to Manning's n uncertainty (surface conveyance to the inlet drives the variability)
- Use the Fiedler gradient at the coupling cell (§7.4) to confirm whether the inlet lies in a high-connectivity region of the mesh

---

6. Advanced configuration

6.1 Spatial Manning's n

Use a spatially-varying Manning's n field when the domain has heterogeneous roughness that you expect to be uncertain at spatial scales comparable to the correlation length (e.g., patchy vegetation or variable pavement condition):

```
[2D_ROM]
ENABLE              YES
MEMBERS             50
MODES               20
MANNINGS_PERT       0.20
MANNINGS_CORR_LEN   30.0    ; 30 m correlation length
```

The generated field has mean 1.0 and standard deviation ≈ `MANNINGS_PERT` (after normalisation). The correlation length should match your best estimate of the spatial coherence of roughness variation (e.g., land-use patch sizes, paving unit widths).

6.2 Auto vs explicit K_eff

`K_EFF AUTO` (the default, triggered by any value ≤ 0) is appropriate for most runs. Set it explicitly if: - The mesh is partially wet at start and the automatic estimate is computed over a non- representative wet region. - You want bit-reproducible results independent of the initial depth distribution. - The AUTO estimate produces unreasonably large spread (K_eff too large → fast decay → spread underestimated; K_eff too small → slow decay → spread overestimated).

To diagnose: enable verbose mode and check the logged K_eff at first seed. Then set it explicitly if needed:

```
[2D_ROM]
K_EFF    8.5    ; m^(4/3)/s — set from logged AUTO value
```

6.3 Reseeding

The ROM seeds from the current deterministic depth field. As the wetting front advances, the seed state becomes stale and spread may be underestimated. `WET_RESEED_FRACTION` controls how aggressively the ROM reseeds:

```
[2D_ROM]
WET_RESEED_FRACTION       0.02   ; reseed when wet area changes by 2%
        of domain
WET_RESEED_MIN_INTERVAL   30.0   ; at most once every 30 s
```

After each reseed, spread briefly collapses to zero (all members restart from the same deterministic state) and then re-grows. If your output frequency is coarser than the reseed interval, this collapse is not visible in the output.

6.4 Parametric tails

With small M (< 30) the sort-based q95 is noisy because only 1–2 samples determine the top percentile. `PARAMETRIC_TAILS YES` fits a log-normal to the wet-cell population and uses the analytic 95th percentile:

```
[2D_ROM]
MEMBERS           20
PARAMETRIC_TAILS  YES
```

The log-normal fit uses method of moments on the log-transformed wet depths. It may overestimate q95 if the true distribution has a lighter tail than log-normal — always verify against a high-M run before relying on parametric tails for design decisions.

---

7. Implementation inventory

This section describes exactly what is implemented in the codebase as of 2026-05-25.

7.1 2D Spectral ROM (SpectralROM) — COMPLETE

Files: `src/engine/2d/uncertainty/SpectralROM.hpp/cpp`
Tests: `tests/unit/engine/test_2d_spectral_rom.cpp` (47/47 pass)

The core ROM struct. Methods:

Method	What it does
`initialize()`	Allocates `a_ensemble[M×k]`, builds LHS design (or uses external samples).
`seed(h_full)`	Projects current deterministic depth onto basis: `a_i = Pᵀ h` for all members.
`advance(dt, K_eff, rainfall, h_cell)`	Advances all members via exact exponential integrator. Optionally uses per-mode Rayleigh K_eff.
`computeQuantiles(parametric_tails)`	Reconstructs per-cell depths, sorts, fills q05/q50/q95.
`applyCouplingFlux(cps, heads, mesh, dt, rom1d)`	Applies per-member orifice flux at each coupling point; updates ROM coefficients; fills `coupling_unc_output`.
`setEnsembleRainfall(rates)`	Replaces scalar rainfall_mult path with per-member runoff rates from RunoffEnsemble.
`setExternalSamples(mann, rain)`	Injects UncertaintyEnsemble's LHS columns instead of building internal ones.
`setCdSamples(cd)`	Injects discharge-coefficient LHS column.

Method	What it does
`is_ready()`	True after initialize() succeeds with a valid basis.

Integration: `CvodeSurfaceSolver` owns a `SpectralROM` instance, seeds it on first advance, calls `advance()` + `computeQuantiles()` after each successful CVODE step, and calls `applyCouplingFluxToROM()` after `computeCouplingExchange()`.

`SurfaceRouter2D::rom()` exposes a const pointer;
`SurfaceRouter2D::couplingOutput()` exposes the per-coupling-point bounds.

7.2 UncertaintyEnsemble — COMPLETE

Files: `src/engine/uncertainty/UncertaintyEnsemble.hpp/cpp`
Tests: via spectral ROM tests

The single LHS owner. Call `generate()` once; then pass column views to consumers:

```
UncertaintyEnsemble ens;
ens.n_members       = 50;
ens.mannings_pert_2d = 0.20;
ens.rainfall_pert_2d = 0.10;
ens.soil_pert        = 0.25;
ens.cd_pert          = 0.10;
ens.generate();

rom.setExternalSamples(ens.manningsSamples2D(),
        ens.rainfallSamples2D());
rom.setCdSamples(ens.cdSamples());
```

Four decorrelated LHS columns: Manning (ascending), rainfall (descending), soil (Fisher-Yates shuffled), Cd (independently shuffled).

7.3 Spatial fields (CorrelatedFieldGenerator) — COMPLETE

Files: `src/engine/2d/uncertainty/CorrelatedFieldGenerator.hpp/cpp`, `src/engine/2d/uncertainty/SpatialUncertaintyField.hpp`
Tests: `tests/unit/engine/test_engine_2d_spatial_field.cpp` (13/13 pass)

Generates M×n_tri fields with exponential spatial correlation. Uses a grid-indexed neighbourhood structure for O(n_tri × avg_neighbours) preprocessing, with `3·corr_len` search radius. Fields are stored in `SpectralROM::spatial_mannings` and `SpectralROM::spatial_rainfall`.

7.4 Fiedler diagnostic (2D + 1D) — COMPLETE

Files: `src/engine/2d/uncertainty/FiedlerDiagnostic.hpp` (header-only), `src/engine/uncertainty/FiedlerDiagnostic1D.hpp` (header-only)
Tests: 7 tests in `test_2d_spectral_rom` (FiedlerDiagnostic suite), 13 tests in `test_engine_spectral_rom1d` (FiedlerDiagnostic1D suite)

The Fiedler vector (second eigenvector of the Laplacian, or first retained mode after null filtering) identifies bottleneck cells/nodes: those where a small perturbation in connectivity most strongly divides the network into two sub-domains. Large Fiedler gradient = high coupling sensitivity.

2D usage:

```
#include "2d/uncertainty/FiedlerDiagnostic.hpp"
FiedlerDiagnostic fd_2d(precond_2d);    // uses P[:,0] (null already
        filtered)
// fd_2d.grad[t] = max |φ₂[t]−φ₂[t']| / dist over face-sharing
        neighbours of t
// fd_2d.rank[0] = index of cell with highest gradient (bottleneck)
double sensitivity = fd_2d.grad[coupling_point.cell_idx];
```

1D usage:

```
#include "uncertainty/FiedlerDiagnostic1D.hpp"
FiedlerDiagnostic1D fd_1d(graph_eigen_basis);
// fd_1d.gradAtFullNode(node_idx, rom1d.full_to_active) → gradient at
        SWMM node
double sensitivity = fd_1d.gradAtFullNode(cp.node_idx,
        rom1d.full_to_active);
```

Cells/nodes with high Fiedler gradient are where coupling uncertainty (§5.6) is most consequential: small changes in Manning's n there produce large changes in flow partition.

7.5 Runoff ensemble (RunoffEnsemble) — COMPLETE

Files: `src/engine/uncertainty/RunoffEnsemble.hpp/cpp`, `src/engine/uncertainty/SoilParameterLHS.hpp`
Tests: `tests/unit/engine/test_runoff_ensemble.cpp` (13/13 pass)

Runs M infiltration trajectories in parallel with independent soil multipliers from `UncertaintyEnsemble::soilSamples()`:

Infiltration model	Perturbed parameter
Green-Ampt	`Ks_i = Ks_base × soil_mult[i]`
Horton	`f0_i = f0_base × mult[i]`, `fmin_i = fmin_base × mult[i]`
Curve Number	`S_i = S_base / soil_mult[i]` (higher mult → more infiltration → less runoff)

The per-member surface runoff rates can be fed to `SpectralROM::setEnsembleRainfall()` to propagate infiltration uncertainty through the 2D surface routing.

7.6 WQ uncertainty bounds (WQUncertaintyBounds) — COMPLETE

File: `src/engine/uncertainty/WQUncertaintyBounds.hpp` (header-only)
Tests: in `test_runoff_ensemble.cpp`

Analytical bounds for first-order constituent decay `c(t) = c₀ exp(−k·t)` with uncertain k:

```
q05_conc = c₀ exp(−k·(1+p)·t)    (highest k → most decay → lowest concentration)
q95_conc = c₀ exp(−k·(1−p)·t)
```

No ODE solve needed. The `p` parameter is taken from `soil_pert` (as a proxy for rate constant uncertainty).

7.7 1D Spectral ROM (SpectralROM1D + GraphEigenBasis) — COMPLETE

Files: `src/engine/uncertainty/GraphEigenBasis.hpp/cpp`, `src/engine/uncertainty/NetworkLaplacian1D.hpp`, `src/engine/uncertainty/SpectralROM1D.hpp/cpp`
Engine hooks: `SWMMEngine::buildROM1D()`, `SWMMEngine::computeK1d()` in `src/engine/core/SWMMEngine.cpp`
Tests: `tests/unit/engine/test_engine_spectral_rom1d.cpp` (39/39) + `tests/unit/engine/test_engine_1d_rom_integration.cpp` (5/5)

A 1D network spectral ROM mirroring the 2D design:

- `GraphEigenBasis` — Lanczos+QL eigensolver for arbitrary CSR-format graphs. Requires n_active_nodes ≥ 4. The first retained mode (after null mode filtering) is the Fiedler mode.
- `NetworkLaplacian1D` — builds the graph Laplacian from conduit-to-node connectivity. Outfall nodes are excluded from the active set.
- `SpectralROM1D` — seed/advance/computeQuantiles on the 1D network. Exposes `full_to_active[]` for mapping SWMM node indices to active ROM indices, and `reconstructHead(member, active_node)` for per-member head at a node (used in 2D coupling).
- `SWMMEngine::buildROM1D()` — called in `initialize()` when any active 1D uncertainty source is configured, or when the 2D ROM is enabled. Builds the Laplacian from `ctx_.links` conduit pairs, runs the eigensolver, seeds from current node heads, and registers with `surface_router_.setROM1D()`.
- `SWMMEngine::computeK1d()` — diffusion-wave diffusivity `D = h^(5/3) / (2n√S)` [m²/s], averaged over active conduits and normalised by mean conduit length squared `L²` to yield 1/s for the dimensionless graph Laplacian eigenvalues.

Accessing 1D quantiles from C++:

```cpp
// After swmm_engine_step() loop:
auto* eng = static_cast<openswmm::SWMMEngine*>(handle);
const auto* rom1d = eng->rom1d();
if (rom1d && rom1d->is_ready()) {
    // rom1d->q05 / q50 / q95 — length n_active_nodes each
    // rom1d->full_to_active[swmm_node_idx] — -1 if outfall
    for (int ai = 0; ai < rom1d->n_nodes; ++ai) {
        double spread = rom1d->q95[ai] - rom1d->q05[ai];
        // ...
    }
}
```

Note — 1D ROM output not in HDF5. The 1D quantiles are currently only accessible via the C++ cast API above. Writing them to the binary output file is deferred (no parser key or plugin hook exists yet).

7.8 2D↔1D Coupling ROM path — COMPLETE

File: `src/engine/2d/uncertainty/SpectralROM.cpp` — `applyCouplingFlux()`

When `SpectralROM1D` is registered via `SurfaceRouter2D::setROM1D()`, the coupling flux computes per-member 1D head via `rom1d->reconstructHead(member, active_idx)` instead of using the single shared deterministic head. This gives each ensemble member a physically consistent pair of (2D depth, 1D head) for the orifice equation.

7.9 Regression and integration tests — COMPLETE

Test	Description	Count
`test_engine_2d_spectral_rom`	ROM unit tests (all paths)	47/47
`test_engine_spectral_rom1d`	1D ROM + GraphEigenBasis + FiedlerDiagnostic1D	39/39
`test_engine_2d_spatial_field`	CorrelatedFieldGenerator	13/13
`test_engine_runoff_ensemble`	RunoffEnsemble + WQ bounds	13/13
`test_engine_regression`	Cross-engine + self-consistency (Example1–3)	6/6
`test_engine_2d_rom_integration`	Full engine lifecycle with 2D ROM	1/1
`test_engine_1d_rom_integration`	Full engine lifecycle with 1D ROM	5/5

7.10 HDF5 output — NOT YET WIRED

`Default2DOutputPlugin` exists at `src/engine/2d/output/Default2DOutputPlugin.cpp` and is designed to write CF-1.11/UGRID-1.0 HDF5 with datasets:

```
/Mesh2_face_depth_q05    [nTime, nFace]
/Mesh2_face_depth_q50    [nTime, nFace]
/Mesh2_face_depth_q95    [nTime, nFace]
```

However, it is **not yet compiled** (not in `OPENSWMM_2D_SOURCES`), HDF5 is not in `vcpkg.json`, and no `2D_OUTPUT_FILE` parser key exists. Until this is wired, read quantiles via the C++ API as shown in §2.1.

---

8. Known limitations

1. **Linear ROM around the seed state.** The Galerkin ROM linearises the diffusion-wave operator at the moment of seeding. If the free surface deforms significantly between seed events, the ROM diverges from the deterministic solution. The `WET_RESEED_FRACTION` guard partially mitigates this but does not fix it for rapidly evolving wetting fronts or hydraulic jumps.

2. **1D ROM is diffusion-wave; DYNWAVE is not.** SWMM's 1D solver integrates the full Saint-Venant equations (continuity + momentum). The 1D ROM propagates uncertainty using a diffusion-wave (Laplacian) approximation, which drops the momentum term. For subcritical, gradually-varying flow in partially-full pipes — the standard urban drainage design case — this is a good approximation. It breaks down for: surcharging or fully-pressurized mains; rapidly rising events where `∂Q/∂t` dominates; tidal or strongly backwater-controlled reaches; pump stations and sluice gates. In those regimes the ROM identifies the right *spatial pattern* of sensitivity but may mis-estimate band width by 30–50%. The 2D CVODE solver uses the diffusion-wave equation by design, so this limitation applies only to the 1D ROM. See §4.6 for the full discussion.

3. **Mean K_eff (not per-cell).** The scalar K_eff path uses a single effective conductance for the full domain. Per-mode Rayleigh-quotient K_eff (§4.8 Option A) partially corrects for this, but for highly non-uniform Manning's n distributions the spread may be systematically underestimated in rougher zones.

4. **Uniform IC → zero ROM spread at t=0.** A spatially uniform initial depth projects entirely onto the null Laplacian eigenvector (constant vector). Spread builds only as the solver creates non-uniform depth patterns (rainfall, coupling, wetting front). This is expected behaviour, not a bug. For zero-spread diagnostics, seed with a non-uniform field or wait at least one SWMM routing step with active rainfall.

5. **Fixed spatial field seeds.** Spatial Manning and rainfall fields use fixed internal seeds (`0xdeadbeef01`, `0xcafebabe02`). Full seed control from the shared ensemble seed is not yet implemented.

6. **1D ROM quantiles not written to output file.** The 1D ROM is now fully wired into the engine (`buildROM1D()` / `computeK1d()` / advance per step). Quantiles are accessible via `SWMMEngine::rom1d()` (see §7.7) but are not yet written to the binary output or any HDF5 file. For now, read them via the C++ cast API after each `swmm_engine_step()` call.

7. **q50 bias vs deterministic output.** For non-zero perturbations the ROM median q50 differs slightly from the deterministic CVODE result due to linearisation. For 0% perturbation (all members identical) q50 matches CVODE exactly by construction. A formal bias validation test (0% perturbation → q50 within 2% of no-ROM output) is on the roadmap.

---

9. Performance

9.1 2D surface mesh (measured)

Benchmark setup: sloped domain, uniform initial depth 0.10 m, 3 cm Gaussian bump, 30 advance steps × dt=1 s. Full MC: 20 perturbed CVODE runs each advancing t=0→30 s.

Configuration	Time
ROM k=6, M=20, 50×50 mesh (5k cells)	1.85 ms
ROM k=10, M=20, 50×50 mesh	2.93 ms
ROM k=10, M=50, 50×50 mesh	4.03 ms
Full MC M=1 (1 CVODE run), 50×50	177 ms
Full MC M=20, 50×50	3 705 ms
ROM k=10, M=20, 100×100 mesh (20k cells)	11.5 ms
Full MC M=1, 100×100	610 ms

ROM vs full MC speedup: ~1265× at 50×50, M=20; ~1060× implied at 100×100.

ROM advance cost scales as O(M · k · n_steps) — purely arithmetic, no ODE solves. Doubling M doubles ROM overhead, which is ~1% of total simulation time at M=50.

9.2 1D sewer network (estimated, N=10,000 nodes)

For a pure 1D sewer network with N=10,000 active nodes, M=50 members, k=20 modes:

Phase	Cost	Notes
Lanczos eigensolver (one-time at `initialize()`)	~15 ms	O(N·k) sparse mat-vec; paid once regardless of simulation length
ROM `advance()` per routing step	< 1 μs	M×k = 1,000 scalar exponentials
`computeQuantiles()` per routing step	~2 ms	M×N = 500,000 reconstructions + per-node sort
Per-step overhead vs deterministic DYNWAVE	< 1%	DYNWAVE Newton solve dominates at ≫ 1 ms/step

For a 6-hour storm at 30-second routing intervals (720 steps):

	Full Monte Carlo (50 DYNWAVE runs)	ROM sidecar
Total overhead	~50× baseline runtime	~0.5% of baseline
Typical wall time	hours	seconds

Quantile reconstruction (M×N per step) is the dominant 1D ROM cost for large networks. For N=10,000 and M=50, each step reconstructs 500,000 head values. At 30 s routing intervals this amounts to roughly 700 ms of quantile work per simulated hour. To

reduce it: lower M, increase the routing interval, or compute quantiles only on output steps rather than every step.

Lanczos build requires n_active ≥ 4 conduit-endpoint nodes (outfalls excluded). Build time scales as O(N·k·iters) where iters ≈ 3k for typical Lanczos convergence. At N=10,000 and k=20 this is approximately 15 ms — a one-time cost absorbed into `swmm_engine_initialize()`.

---

10. Visualization

10.1 Python scripts (scripts/uncertainty/)

Script	Output
`plot_coupled_1d_2d.py`	6-panel: 2D q50/spread maps + per-node flow hydrographs with q05/q50/q95 bands at 4 nodes along the 1D trunk
`plot_pipe_transect.py`	4-panel: longitudinal flow profile at 4 time snapshots, line colored by relative spread (%) using `LineCollection`
`plot_wet_event_bands.py`	1D KW confidence bands (Edinburgh trunk, ±25% Manning)
`plot_2d_uncertainty_maps.py`	2D spatial maps of q50 depth and q95−q05 spread
`plot_coupling_uncertainty_5A_5B.py`	Per-coupling-point flux bounds over time
`fiedler_diagnostic.py`	Fiedler vector analysis — identifies bottleneck nodes
`mcmc_vs_rom.py`	ROM prior vs MCMC posterior comparison

Run with the Anaconda Python:

```
MPLBACKEND=Agg /Users/corinnewiesner/anaconda3/bin/python3 \
    scripts/uncertainty/plot_pipe_transect.py --out transect.png
```

10.2 Colored pipe transect

The `plot_pipe_transect.py` script uses `matplotlib.collections.LineCollection` to draw each segment of a pipe colored by its relative uncertainty:

```
from matplotlib.collections import LineCollection
import matplotlib.colors as mcolors

norm = mcolors.Normalize(vmin=0, vmax=50)     # 0–50% relative spread
pts  = np.column_stack([x_station, q50_flow])
segs = np.stack([pts[:-1], pts[1:]], axis=1)
lc   = LineCollection(segs, cmap="YlOrRd", norm=norm, linewidth=2.5)
```

```
lc.set_array(0.5 * (spread_pct[:-1] + spread_pct[1:]))
ax.add_collection(lc)
fig.colorbar(lc, ax=ax, label="Relative spread (q95−q05)/q50 %")
```

Replace `x_station`, `q50_flow`, and `spread_pct` with arrays from your node extraction loop.

10.3 Future: HDF5 / ParaView / QGIS

Once the HDF5 output pipeline is wired (§7.10), the CF-1.11/UGRID-1.0 datasets will be readable by any CF-aware tool:

```
import xarray as xr
ds = xr.open_dataset("results.h5", engine="netcdf4")
spread = ds["Mesh2_face_depth_q95"] - ds["Mesh2_face_depth_q05"]
```

For ParaView: open the `.h5` file, select `Mesh2_face_depth_q95` and subtract `Mesh2_face_depth_q05` via a Calculator filter to visualise the uncertainty band width.